

## Otonom Veri Temizleme ve Analiz Aracı: Kapsamlı Proje Geliştirme ve Mimari Dokümantasyonu (Blueprint)

Bu doküman, kullanıcıların ham veri dosyalarını yüklediği, LLM (Büyük Dil Modeli) entegrasyonu ile bu verinin analiz edilerek temizlendiği, verinin doğasına göre uygun makine öğrenmesi görevini (Sınıflandırma veya Regresyon) otomatik olarak belirleyip görselleştiren ve nihayetinde kullanıma hazır, temiz bir CSV dosyası sunan web tabanlı uygulamanın tam kapsamlı teknik ve operasyonel yol haritasıdır. İlk konsept tasarımı ile detaylı takım mimarisinin birleştirilmiş ve genişletilmiş halidir.

## 1. Ekip Roller ve İş Dağılımı

Projenin paralel, hızlı ve hatasız geliştirilebilmesi için modüler bir iş paylaşımı ve net rol tanımları şarttır:

- Frontend Geliştirici (UI/UX):** Kullanıcı arayüzünü geliştirir. Sürükle-bırak dosya yükleme bileşenleri, asenkron işlemler (backend'den yanıt beklerken) için ilerleme çubukları (progress bar), yükleme animasyonları ve analiz sonuçlarının dinamik, etkileşimli grafiklerle (Plotly.js, Echarts) gösterilmesinden sorumludur. Ayrıca kullanıcıya veriyi manipüle etme (LLM kararlarını ezme) arayüzü sunar.
- Backend Geliştirici:** Dosya yükleme, indirme ve analiz API'lerini (RESTful) yazar. Uygulamanın bel kemiğini oluşturur. Uzun süren veri analiz işlemlerini yönetmek için asenkron görev kuyruklarını (örn. Celery, Redis) kurar. Güvenlik, dosya boyutu limitleri ve CORS ayarlarını yönetir.
- AI & Veri Mühendisi:** Sistemin "beynidir". LLM prompt mühendisliği, LangChain/LlamaIndex entegrasyonu, Pandas/Polars ile veri temizleme betiklerinin yazılması ve makine öğrenmesi (sınıflandırma/regresyon) tespiti için karar motorunu (agent) geliştirir. LLM'in ürettiği kararları koda/fonksiyona döker.
- DevOps (Ortak Sorumluluk):** Projenin Docker ve Docker Compose ortamına alınması, veritabanı/Redis kurulumları, CI/CD pipeline'larının oluşturulması ve projenin bir bulut sunucuya (AWS EC2, GCP, DigitalOcean) dağıtılması süreçlerini yönetir.

## 2. Önerilen Teknoloji Yığını ve Sistem Mimarisi

Uygulama, eşzamanlı kullanıcı isteklerini kaldırabilmesi ve gigabaytlarca büyüklükteki veri

setlerini işlerken kilitlenmemesi (timeout almaması) için tam asenkron bir mimaride tasarlanmıştır.

| Katman                              | Önerilen Teknoloji                   | Görevi ve Teknik Detaylar                                                                                                                                                                                                                  |
|-------------------------------------|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Kullanıcı Arayüzü (Frontend)</b> | React.js (Next.js) veya Streamlit    | Hızlı bir prototip (PoC) ve test süreci için <b>Streamlit</b> ; ancak üretim ortamına (production) çıkacak profesyonel, ölçeklenebilir ve özelleştirilebilir bir ürün için <b>React.js / Next.js</b> kesinlikle önerilir.                  |
| <b>API Ağ Geçidi (Backend)</b>      | FastAPI (Python)                     | Yüksek performanslı, asenkron destekli RESTful API sunar. Veri doğrulama (Pydantic ile), yetkilendirme ve dosya chunk (parçalı) yükleme işlemleri burada yönetilir.                                                                        |
| <b>Asenkron Görev Kuyruğu</b>       | Celery + Redis / RabbitMQ            | Büyük CSV/Excel dosyalarının analizi ve LLM çağrıları vakit alacağı için (bazen dakikalar sürebilir), HTTP request'ini bekletmemek adına işlemler arka planda worker'lara (işçilere) devredilir. Frontend bu işlemlerin durumunu sorgular. |
| <b>AI ve Veri İşleme Motoru</b>     | Pandas/Polars, LangChain, Gemini API | Büyük veri setleri için Pandas yerine belleği daha iyi yöneten <b>Polars</b> tercih edilebilir. LLM ile iletişim kuran ajanların (agents) bulunduğu çekirdek katmandır.                                                                    |
| <b>Geçici Depolama</b>              | AWS S3, MinIO veya Yerel Disk        | Kullanıcının yüklediği ham dosyalar ve sistemin ürettiği temizlenmiş dosyalar, işlem bitene kadar burada geçici olarak (TTL policy ile) saklanır.                                                                                          |

### 3. Sistemin Detaylı Çalışma Akışı (Data Flow Pipeline)

- Veri Yükleme ve API Etkileşimi (Upload):** Kullanıcı dosyayı arayüzden yükler. Frontend, dosyayı backend'in yükleme endpoint'ine gönderir. Dosya geçici depolama alanına kaydedilir.

2. **Metadata Çıkarımı (Veri Profileleme):** Backend, dosyanın tamamını LLM'e GÖNDERMEZ (Bu çok kritik bir kuraldır). Sadece veri şemasını okur: Kolon adları, veri tipleri (int, float, string), her kolondaki eksik değer (NaN) sayıları, benzersiz değer sayıları (nunique) ve ilk 5-10 satırlık veri örneğini (metadata) çıkarır. Backend bu aşamada görevi kuyruğa atar ve frontend'e bir job\_id döner.
3. **LLM ile Otonom Karar Mekanizması (AI Agent):** Çıkarılan metadata, LangChain/Llamaindex üzerinden yapılandırılmış bir prompt ile LLM'e gönderilir. LLM veriyi analiz eder ve sistemin anlayabileceği bir JSON reçetesi oluşturur. (Hangi kolon hedef? Hangi kolon silinmeli? Boşluklar nasıl dolmalı?)
4. **Veri İşleme ve Temizleme (Data Execution):** LLM'den dönen JSON reçetesi, arka planda çalışan Python fonksiyonları tarafından parse edilir. Ham veri üzerinde fiziksel temizlik işlemleri (drop, fillna, label encoding vb.) gerçekleştirilir ve yeni, temiz DataFrame oluşturulur.
5. **Görselleştirme Hazırlığı:** Temizlenmiş veri seti üzerinden, LLM'in belirlediği problem türüne (Sınıflandırma veya Regresyon) uygun grafiklerin koordinatları hesaplanır. Sınıflandırma için dağılım frekansları, Regresyon için korelasyon matrisleri hazırlanır.
6. **Sonuç Sunumu (Polling & İndirme):** Frontend, elindeki job\_id ile belirli aralıklarla backend'e sorar (Long polling veya WebSocket). İşlem tamamlandığında backend; analiz raporunu, çizilecek grafiklerin JSON verisini ve temizlenmiş dosyanın indirme bağlantısını (download link) frontend'e iletir.

## 4. API Tasarımı (Backend - Frontend İletişimi Sözleşmesi)

Ekiplerin birbirini beklemeden paralel geliştirme yapabilmesi için API uç noktalarının (endpoints) şu yapıda olması önerilir:

**POST /api/v1/analyze**

- İstek Tipi: multipart/form-data (Yüklenen veri dosyası)

- Başarılı Yanıt (202 Accepted):

```
{
  "job_id": "9876-abcd-1234",
  "status": "queued",
  "message": "Dosya kuyruğa eklendi."
}
```

**GET /api/v1/status/{job\_id}**

- Devam Eden İşlem Yanıtı (200 OK):

```
{
```

```
"status": "processing",
"progress": 45,
"message": "LLM veriyi inceliyor ve temizleme stratejisi
oluřturuluyor..."
}
- Tamamlanan İşlem Yanıtı (200 OK):
{
  "status": "completed",
  "download_url": "/api/v1/download/9876-abcd-1234",
  "ai_decisions": {
    "task_type": "Regression",
    "target_column": "House_Price",
    "applied_steps": [
      "'Age' kolonundaki boşluklar ortalama ile dolduruldu.",
      "'ID' ve 'Name' kolonları analiz dışı olduđu için silindi."
    ]
  },
  "chart_data": {
    // Plotly veya Echarts'ın doğrudan tüketebileceği grafik datası
  }
}
```

## 5. LLM Prompt Mimarisi (Geliřmiř JSON Formatı)

Sistemin kararlı çalışabilmesi için LLM'in kesinlikle serbest metin (düz yazı) değıl, makinece okunabilir yapılandırılmış veri (JSON) döndürmesi zorunludur. LangChain'in StructuredOutputParser özelliğı kullanılmalıdır.

**Sistem Rolü:** Sen kıdemli bir veri mühendisi ve makine Öğrenmesi uzmanısın.  
**Görev:** Ařağıda verilen veri řemasına ve özetine bakarak, otonom veri temizleme adımlarını ve makine Öğrenmesi görev türünü belirle.

### KATI KURALLAR:

- YALNIZCA geçerli bir JSON objesi döndür. Markdown etiketleri (```json vb.) kullanma, açıklama yapma.
- 'target\_column' veri řemasında kesinlikle var olan bir sütun olmalıdır.
- 'missing\_value\_strategy' için sadece pandas'ta uygulanabilir metotlar kullan ('mean', 'median', 'mode', 'drop').

**Veri Şeması ve İlk 5 Satır:**

```
{DATA_METADATA_BURAYA_GELECEK}
```

#### Beklenen JSON Çıktı Formatı:

```
{
  "target_column": "TahminEdilecekSutunAdi",
  "task_type": "Classification" | "Regression",
  "columns_to_drop": ["gereksiz_sutun1", "gereksiz_sutun2"],
  "missing_value_strategy": {
    "yas_sutunu": "median",
    "maas_sutunu": "mean"
  },
  "reasoning": "Bu kararları neden aldığını açıklayan 1-2 cümlelik kısa
  metin."
}
```

## 6. İşbirliği ve Git Stratejisi (Git Flow)

Ekibin kodları ezmesini (merge conflict) önlemek için standart süreçler izlenmelidir:

- main: Sadece üretim (production) ortamında çalışan, tamamen test edilmiş stabil kodlar bulunur.
- develop: Ortak geliştirme dalıdır. Ekiplerin işleri burada birleştirilir ve test ortamına çıkarılır.
- feature/backend-api: Backend geliştiricinin çalıştığı özel branch.
- feature/ai-engine: AI/Veri mühendisinin Langchain denemelerini yaptığı branch.
- feature/ui-dashboard: Frontend geliştiricisinin arayüzü kodladığı branch.

## 7. Karşılaşılabilecek Zorluklar, Performans ve Güvenlik Uyarıları

| Risk / Zorluk                                    | Çözüm Stratejisi                                                                                                                                                                                                                        |
|--------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Büyük Veri Yönetimi (OOM - Out of Memory)</b> | Pandas veriyi tek seferde RAM'e yükler. 1-2 GB'lık CSV'ler sunucuyu çökertebilir. Backend tarafında dosyalar <i>chunk</i> 'lar halinde okunmalı veya <b>Polars</b> / <b>Dask</b> gibi büyük veri destekli kütüphaneler kullanılmalıdır. |

| Risk / Zorluk                               | Çözüm Stratejisi                                                                                                                                                                                                                                                                           |
|---------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Veri Gizliliği (KVKK / GDPR)</b>         | LLM API'lerine (Gemini vb.) <b>kesinlikle</b> verinin tamamı veya kişisel veriler (isim, TC, email) gönderilmemelidir. Sadece anonimleştirilmiş kolon adları ve sayısal istatistikler gönderilmelidir.                                                                                     |
| <b>LLM Halüsinasyonları</b>                 | LLM, regresyon olması gereken bir hedefe sınıflandırma diyebilir veya var olmayan bir kolon adı uydurabilir (Halüsinasyon). Arayüze kesinlikle <b>"Manuel Düzenleme (Override)"</b> butonu eklenmeli, kullanıcı analiz başlamadan önce LLM'in kararlarını ekrandan değiştirebilmelidir.    |
| <b>Kod Çalıştırma Güvenliği (RCE Riski)</b> | LLM'e doğrudan "Benim için Python kodu yaz" deyip bu kodu sunucuda eval() veya exec() ile çalıştırmak inanılmaz büyük bir güvenlik açığıdır. LLM sadece JSON kuralları üretmeli, kod çalıştırma işlemi sizin önceden yazdığınız güvenli Python/Pandas metotları üzerinden tetiklenmelidir. |

## 8. Proje Fazları (Yol Haritası)

- **Faz 1 (Altyapı ve PoC):** Ekibin kurulması, GitHub repolarının açılması. FastAPI ile boş endpoint'lerin ayağa kaldırılması ve Streamlit/React ile statik dosya yükleme arayüzünün yapılması. (Tahmini: 1 Hafta)
- **Faz 2 (AI Entegrasyonu):** LangChain/LLM karar motorunun yazılması. Yüklenen CSV verisinin Pandas ile özetinin çıkarılıp LLM'den başarılı bir JSON çıktısı alınmasının sağlanması. (Tahmini: 1-2 Hafta)
- **Faz 3 (Veri İşleme ve Asenkron Mimari):** Celery ve Redis kurularak işlemlerin arka plana alınması. LLM'den gelen JSON'a göre verinin fiziksel olarak temizlenmesi kodlarının yazılması. (Tahmini: 2 Hafta)
- **Faz 4 (Görselleştirme ve Export):** Temizlenen verinin sonuçlarının grafikleştirilmesi (Plotly) ve CSV olarak indirme mekanizmasının son kullanıcıya sunulması. Hata testleri. (Tahmini: 1 Hafta)

*Bu doküman ekibinizin proje geliştirme döngüsü boyunca bir rehber niteliği taşıması amacıyla oluşturulmuştur. Başarılar dileriz!*